

Лекция 3

Модели разработки ПО и анализ требований

Модель разработки ПО (Software Development Model, SDM) — структура, систематизирующая различные виды проектной деятельности, их взаимодействие и последовательность в процессе разработки ПО. [Куликов]

Мы рассмотрим следующие модели разработки:

1. Водопадная модель (waterfall model)
2. V-образная модель (V-model)
3. Итерационная инкрементальная модель (iterative model, incremental model) - также можно использовать их по отдельности
4. Спиральная модель (spiral model)
5. Гибкая модель (agile model), в частности Scrum

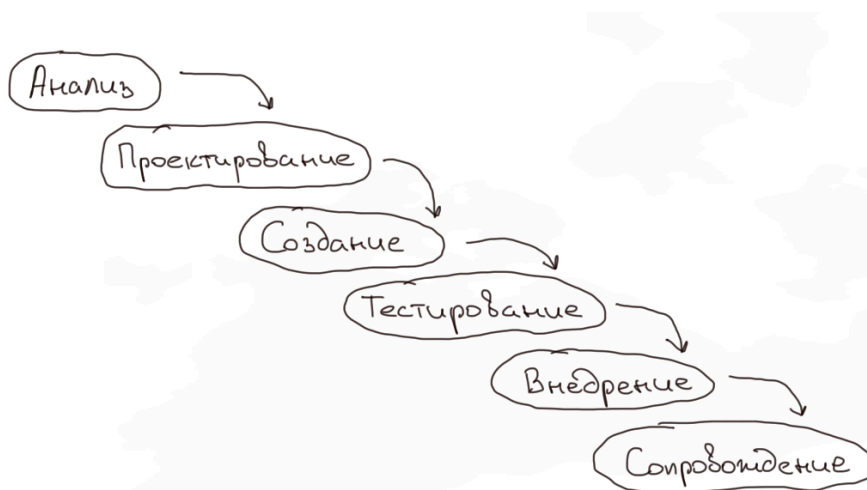
Это не весь перечень доступных моделей и все модели можно разделить на две больших группы: **последовательные** и **итерационные** модели.

На текущий момент **чаще всего используются гибкие модели разработки**, в основе которых лежит итерационная инкрементальная модель.

Каскадная модель

Waterfall Model или «водопад»

В этой модели разработка осуществляется поэтапно: каждая следующая стадия начинается только после того, как заканчивается предыдущая.



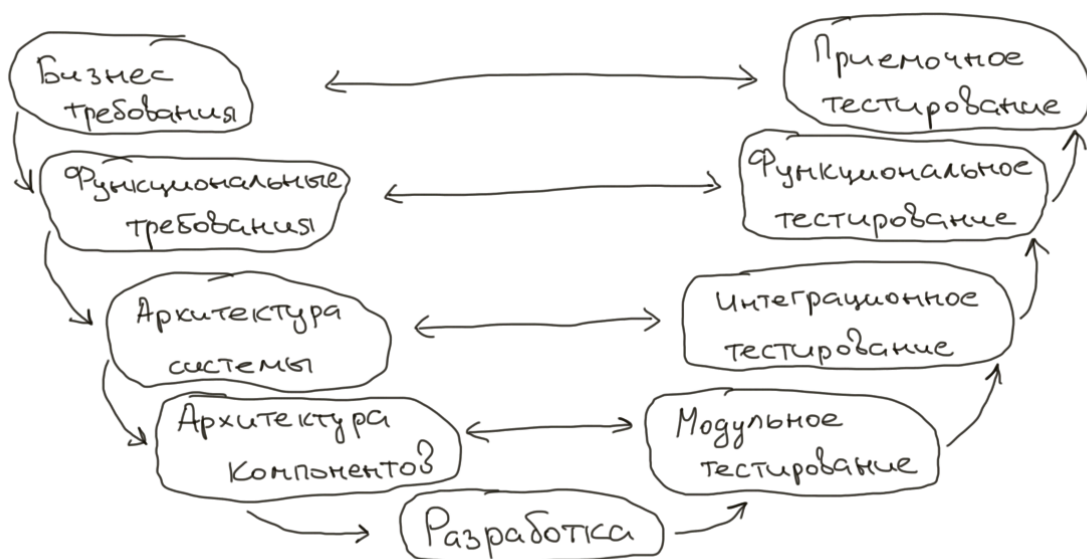
Преимущества:

- Линейность модели облегчает ее понимание;
- Все спецификации и результаты изложены до начала разработки;
- Строгость этапов позволяет планировать сроки завершения всех работ и соответствующие ресурсы
- Хорошо работает для небольших проектов;
- Стоимость проекта определяется на начальном этапе;

Недостатки:

- Сложности при формулировке четких требований и невозможность их изменения;
- Модель водопада неприменима к проектам, требующим непрерывной доработки;
- Отсутствие работающего продукта до окончания последней стадии разработки;
- Тестирование начинается только с середины развития проекта;
- Много технической документации

V-образная модель



V-Model (разработка через тестирование)

Суть этой модели состоит в том, что процессы на всех этапах контролируются, чтобы убедиться в возможности перехода на следующий уровень. Уже на стадии написания требований начинается процесс тестирования. Частично устраняет недостатки каскадной модели.

Преимущества:

- Этапы строго определены;
- Минимизация рисков и устранение потенциальных проблем за счет раннего тестирования;
- Усовершенствованный тайм-менеджмент.

Недостатки:

- Нет возможности адаптироваться к измененным требованиям заказчика;
- Длительное время разработки (иногда до нескольких лет) приводит к тому, что продукт может быть уже не нужен заказчику, поскольку его потребности меняются;
- Нет действий, направленных на анализ рисков.

Итерационная модель



Iterative Model - Итерационная (итеративная) модель

Предполагает разбиение проекта на части (этапы, итерации) и прохождение этапов жизненного цикла на каждом их них. Каждый этап является законченным сам по себе, совокупность этапов формирует конечный результат.

Преимущества:

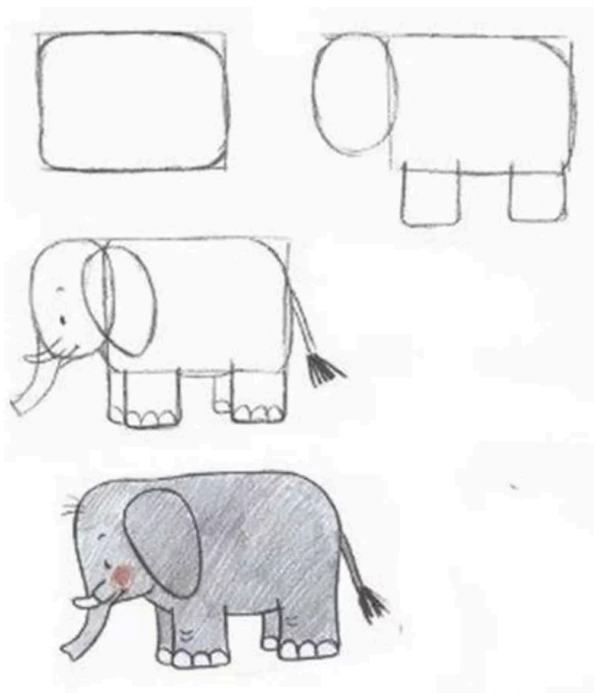
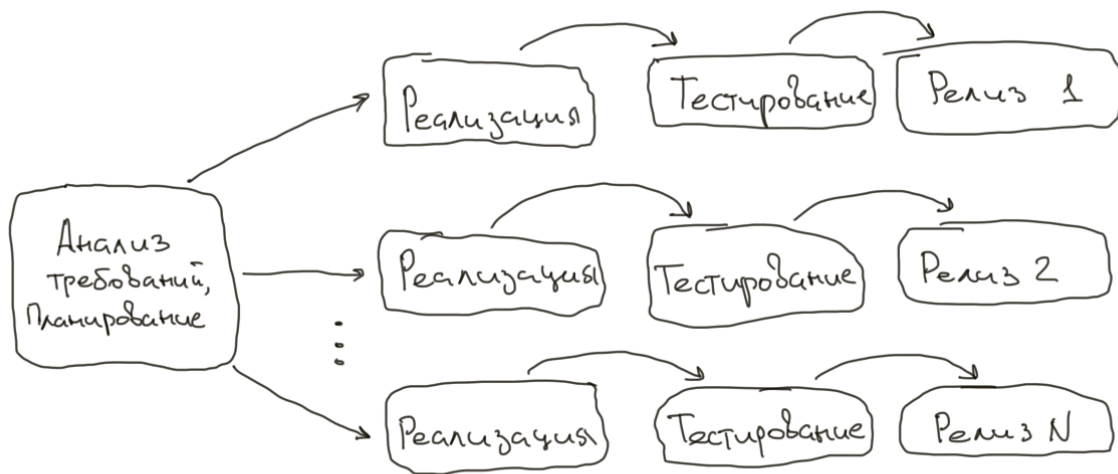
- Быстрый выпуск минимального продукта даёт возможность оперативно получать обратную связь от заказчика и пользователей;
- Фокус на наиболее важных функциях ПО и улучшение их в соответствии с требованиями рынка и пожеланиями клиента.

- Постоянное тестирование пользователями позволяет быстро обнаруживать и устранять ошибки.

Недостатки:

- Вероятность того, что на определенной итерации придётся переписывать большую часть приложения.
- Отсутствие фиксированного бюджета и сроков.

Инкрементная модель



Incremental Model

Эта модель разработки дает возможность делать продукт по частям — инкрементам. Каждая часть представляет собой готовый фрагмент итогового продукта, который в идеале не переделывается. Улучшение продукта проходит запланировано все время пока жизненный цикл разработки ПО не завершится.

Требования к системе определяются в самом начале работы, после чего процесс разработки проводится в виде последовательности версий, каждая из которых является законченным и работоспособным продуктом.

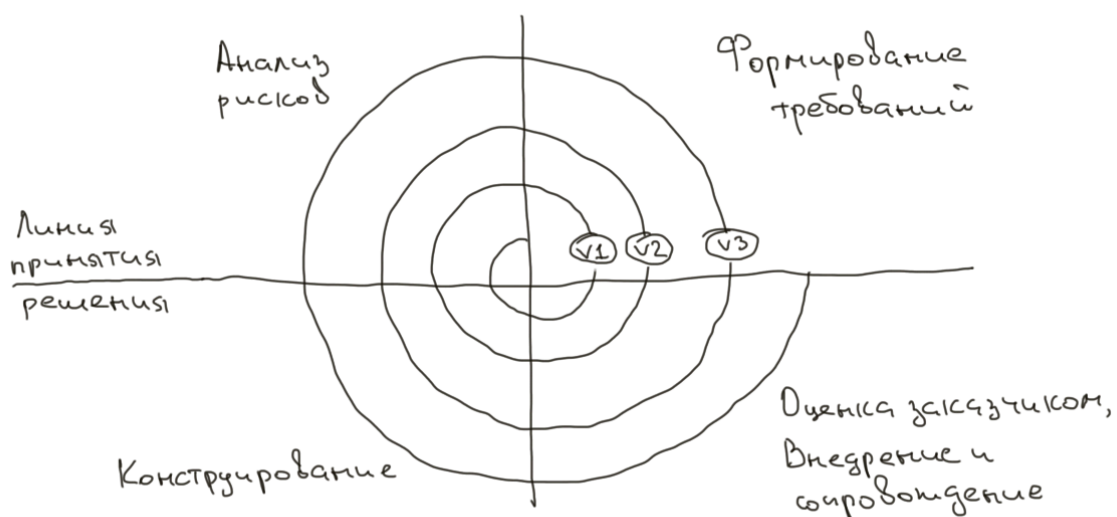
Преимущества:

- Заказчик может дать свой отзыв касательно каждой версии продукта;
- Есть возможность пересмотреть риски, которые связаны с затратами и соблюдением графика;
- Ошибка обходится дешевле.

Недостатки:

- Функциональная система должна быть полностью определена в начале жизненного цикла для выделения итераций;
- При постоянных изменениях структура системы может быть нарушена;
- Сроки сдачи системы могут быть затянуты из-за ограниченности ресурсов (исполнители, финансы).

Спиральная модель



Spiral Model

Эту модель начали использовать в 1988 году. В спиральной модели жизненный путь разрабатываемого продукта изображается в виде спирали, которая, начавшись на этапе планирования, раскручивается с прохождением каждого следующего шага. Таким образом, на выходе из очередного витка:

- получаем готовый протестированный прототип, который дополняет существующую сборку;
- принимается решение, продолжать ли проект.

Спиральная модель предполагает 4 этапа для каждого витка:

1. планирование;
2. анализ рисков;
3. конструирование;
4. оценка результата и при удовлетворительном качестве переход к новому витку.

Преимущества:

- Управлению рисками уделяется особое внимание;
- Дополнительные функции могут быть добавлены на поздних этапах;
- Есть возможность гибкого проектирования.

Недостатки:

- Оценка рисков на каждом этапе является довольно затратной;
- Постоянные отзывы и реакция заказчика может провоцировать все новые и новые итерации, которые могут приводить к временному затягиванию разработки продукта;
- Более применима для больших проектов.

Модель хаоса

Chaos model

Её создатель Л.Б.С.Раун отмечает, что такие модели управления проектами, как спиральная модель и каскадная модель, хотя и хороши в управлении расписаниями и персоналом, не обеспечивают методами устранения ошибок и решениями других технических задач, не помогают ни в управлении конечными сроками, ни в реагировании на запросы клиентов. Модель хаоса — это инструмент пытающийся помочь понять эти ограничения и восполнить пробелы.

Стратегия хаоса — это стратегия разработки программного обеспечения, основанная на модели хаоса. **Главное правило — это всегда решать наиболее важную задачу первой.**

Гибкая (Agile) модель

Agile Model - гибкая модель разработки, по которой сегодня работает большинство ИТ-проектов. Представляет собой совокупность различных подходов к разработке ПО.

Основные идеи Agile:

- люди и взаимодействие важнее процессов и инструментов;
- работающий продукт важнее исчерпывающей документации;
- сотрудничество с заказчиком важнее согласования условий контракта;
- готовность к изменениям важнее следования первоначальному плану.

Преимущества:

- после каждой итерации заказчик может наблюдать результат и понимать, удовлетворяет он его или нет.
- быстрое принятие решений за счет постоянных коммуникаций;
- минимизация рисков;
- облегченная работа с документацией.

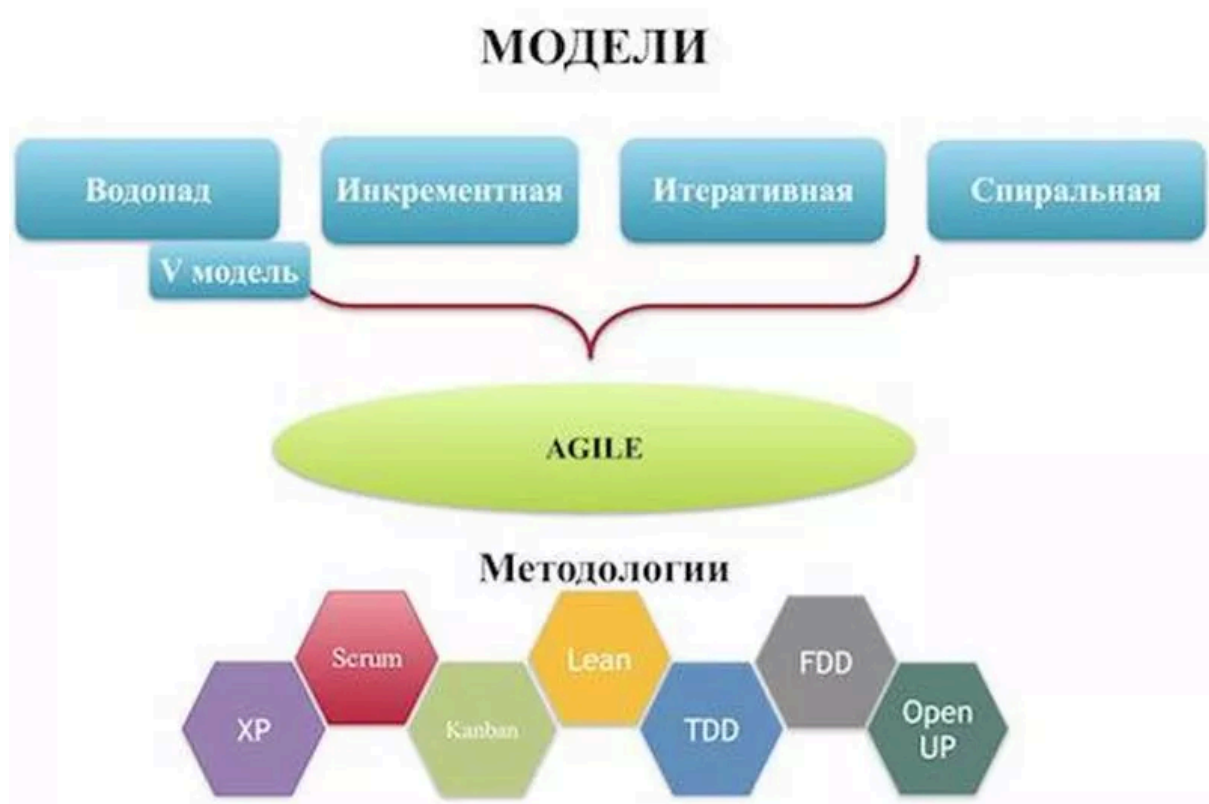
Недостатки:

- большое количество митингов и бесед, что может увеличить время разработки продукта;
- сложно планировать процессы, так как требования постоянно меняются;
- редко используется для реализации больших проектов.

Гибкие методологии разработки ПО

- SCRUM
- Kanban (看板)
- RUP (Rational Unified Process)
- OpenUP
- DSDM (Dynamic Systems Development Model)
- RAD (Rapid Application Development)
- XP (Extreme Programming)
- FDD (Feature Driven Development)
- ASD (Adaptive Software Development)
- DevOps (Development and Operations)
- DAD (Disciplined Agile Delivery)
- Lean SD (Lean Software Development)
- MDD (Model-Driven Development)
- MSF (Microsoft Solutions Framework)
- PSP (Personal Software Process)

- SAgile (Scaled Agile Framework)
- UP (Unified Process)



Анализ требований

Источники и пути выявления требований

Требование (requirement) - условие, которое включает обязательные для выполнения критерии *[ISTQB Glossary]*

Перед тем как к команде разработки попадают финальные требования их необходимо собрать у заказчика. Для этого используют ряд техник на рисунке ниже.

Чаще всего этим занимаются бизнес-аналитики или владельцы продукта.



Рисунок 2.2.d — Основные техники сбора и выявления требований

Уровни и типы требований

Давайте сначала ознакомимся с этой "страшной" схемой, а затем поговорим только про самые важные ее части.

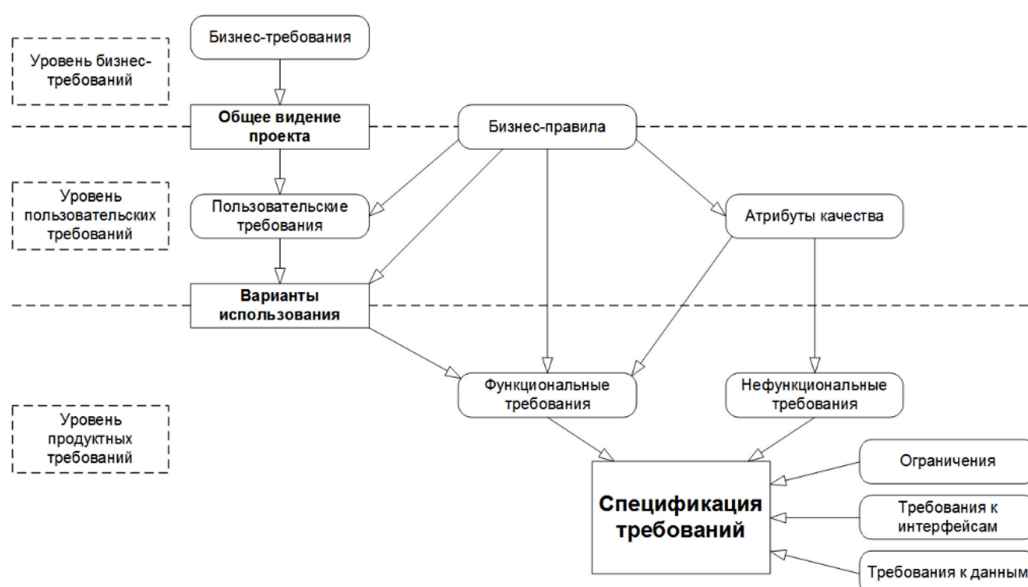


Рисунок 2.2.e — Уровни и типы требований

Бизнес-требования (business requirements) выражают цель, ради которой разрабатывается продукт (зачем вообще он нужен, какая от него ожидается польза, как заказчик с его помощью будет получать прибыль).

Примеры:

- Нужен инструмент, в реальном времени отображающий наиболее выгодный курс покупки и продажи валюты;

- Необходимо в два-три раза повысить количество заявок, обрабатываемых одним оператором за смену;
- Нужно автоматизировать процесс выписки товарно-транспортных накладных на основе договоров.

Пользовательские требования (user requirements) описывают задачи, которые пользователь может выполнять с помощью разрабатываемой системы (реакцию системы на действия пользователя, сценарии работы пользователя).

Пользовательские требования оформляются в виде вариантов использования (use cases), пользовательских историй (user stories), пользовательских сценариев (user scenarios). Часть из них мы рассмотрим далее в уроке.

Примеры:

- При первом входе пользователя в систему должно отображаться лицензионное соглашение;
- Администратор должен иметь возможность просматривать список всех пользователей, работающих в данный момент в системе;
- При первом сохранении новой статьи система должна выдавать запрос на сохранение в виде черновика или публикацию.

Функциональные требования (functional requirements) описывают поведение системы, т.е. ее действия (вычисления, преобразования, проверки, обработку и т.д.).

Примеры:

- В процессе инсталляции приложение должно проверять остаток свободного места на целевом носителе;
- Система должна автоматически выполнять резервное копирование данных ежедневно в указанный момент времени;
- Электронный адрес пользователя, вводимый при регистрации, должен быть проверен на соответствие требованиям RFC822.

Нефункциональные требования (non-functional requirements) описывают свойства системы (удобство использования, безопасность, надежность,

расширяемость и т.д.), которыми она должна обладать при реализации своего поведения.

Примеры:

- При одновременной непрерывной работе с системой 1000 пользователей, минимальное время между возникновением сбоев должно быть более или равно 100 часов;
- Ни при каких условиях общий объем используемой приложением памяти не может превышать 2 ГБ;
- Размер шрифта для любой надписи на экране должен поддерживать настройку в диапазоне от 5 до 15 пунктов.

Спецификация требований (software requirements specification, SRS84) объединяет в себе описание всех требований уровня продукта и может представлять собой весьма объёмный документ (сотни и тысячи страниц).

Неявные требования

Не всегда требования будут описаны на проекте в виде спецификации или пользовательской истории. В таком случае необходимо изучать неявные требования из других источников:

1. Законы, регламенты, инструкции
2. Список задач, существующие тесты и баг-репорты
3. Руководство пользователя
4. Реклама продукта
5. Интервью с командой и заказчиками
6. Чаты и email-переписка
7. Прототип, дизайн-макет
8. Конкурентный анализ, личный опыт

Свойства качественных требований

Завершенность (completeness)

Требование является полным и законченным с точки зрения представления в нём всей необходимой информации, ничто не пропущено по соображениям «это и так всем понятно».

Примеры:

1. Пароли должны храниться в зашифрованном виде
2. Экспорт осуществляется в форматы PDF, PNG и т.д.
3. Приведённые ссылки неоднозначны (например: «см. выше» вместо «см. раздел 123.45.b»)

Атомарность, единичность (atomicity)

Требование является атомарным, если его нельзя разбить на отдельные требования без потери завершённости и оно описывает одну и только одну ситуацию.

Примеры:

1. Кнопка “Restart” не должна отображаться при остановленном сервисе, окно “Log” должно вмещать не менее 20-ти записей о последних действиях пользователя
2. Если пользователь подтверждает заказ и редактирует заказ или откладывает заказ, должен выдаваться запрос на оплату
3. Когда пользователь входит в систему, ему должно отображаться приветствие; когда пользователь вошёл в систему, должно отображаться имя пользователя; когда пользователь выходит из системы, должно отображаться прощание

Непротиворечивость, последовательность (consistency)

Требование не должно содержать внутренних противоречий и противоречий другим требованиям и документам.

Примеры:

1. После успешного входа в систему пользователя, не имеющего права входить в систему...
2. “712.a Кнопка “Close” всегда должна быть красной” и “36452.x Кнопка “Close” всегда должна быть синей” - противоречия в разных требованиях
3. “В случае, если разрешение окна составляет менее 800x600...” — разрешение есть у экрана, у окна есть размер

Недвусмысленность (unambiguousness, clearness)

Требование должно быть описано без использования жаргона, неочевидных аббревиатур и расплывчатых формулировок, должно допускать только однозначное объективное понимание и быть атомарным в плане невозможности различной трактовки сочетания отдельных фраз.

Примеры:

1. «Доступ к ФС осуществляется посредством системы прозрачного шифрования» и «ФС предоставляет возможность фиксировать сообщения в их текущем состоянии с хранением истории всех изменений» - что такое ФС?
2. «Система конвертирует входной файл из формата PDF в выходной файл формата PNG» - некоторые вещи опущены, так как автор считает их очевидными

Выполнимость (feasibility)

Требование должно быть технологически выполнимым и реализуемым в рамках бюджета и сроков разработки проекта.

Примеры:

1. Анализ договоров должен выполняться с применением искусственного интеллекта, который будет выносить однозначное корректное заключение о степени выгоды от заключения договора - технически невыполнимо на данный момент
2. Система поиска должна заранее предусматривать все возможные варианты поисковых запросов и кэшировать их результаты - нельзя реализовать
3. «озолочение» (gold plating) — требования, которые крайне долго и/или дорого реализуются и при этом практически бесполезны для конечных пользователей

Обязательность, нужность (obligatoriness) и актуальность (up-to-date).

Если требование необязательное, оно должно быть просто исключено из набора требований. Если требование нужное, но «не очень важное», для указания этого факта используется указание приоритета. Также исключены (или переработаны) должны быть неактуальные требования

Примеры:

1. Требование было добавлено «на всякий случай», хотя реальной потребности в нём не было и нет.
2. Требованию выставлены неверные значения приоритета по критериям важности и/или срочности.
3. Требование устарело, но не было переработано или удалено.

Прослеживаемость (traceability)

Вертикальная позволяет соотносить между собой требования на различных уровнях требований, горизонтальная позволяет соотносить требование с тест-планом, тест-кейсами, архитектурными решениями и т.д.

Примеры:

1. Требования не пронумерованы, не структурированы, не имеют оглавления, не имеют работающих перекрёстных ссылок.

2. При разработке требований не были использованы инструменты и техники управления требованиями.
3. Набор требований неполный, носит обрывочный характер с явными «пробелами».

Модифицируемость (modifiability)

Это свойство характеризует простоту внесения изменений в отдельные требования и в набор требований.

Примеры:

1. Требования неатомарны и непрослеживаемы, а потому их изменение с высокой вероятностью порождает противоречивость.
2. Требования изначально противоречивы. В такой ситуации внесение изменений (не связанных с устранением противоречивости) только усугубляет ситуацию, увеличивая противоречивость и снижая прослеживаемость.
3. Требования представлены в неудобной для обработки форме (например, не использованы инструменты управления требованиями, и в итоге команде приходится работать с десятками огромных текстовых документов).

Проранжированность по важности, стабильности, срочности (ranked for importance, stability, priority)

Важность характеризует зависимость успеха проекта от успеха реализации требования. Стабильность характеризует вероятность того, что в обозримом будущем в требование не будет внесено никаких изменений.

Срочность определяет распределение во времени усилий проектной команды по реализации того или иного требования.

Корректность (correctness) и проверяемость (verifiability)

Эти свойства вытекают из соблюдения всех вышеперечисленных. В дополнение можно отметить, что проверяемость подразумевает возможность создания объективного тест-кейса (тест-кейсов), однозначно показывающего,

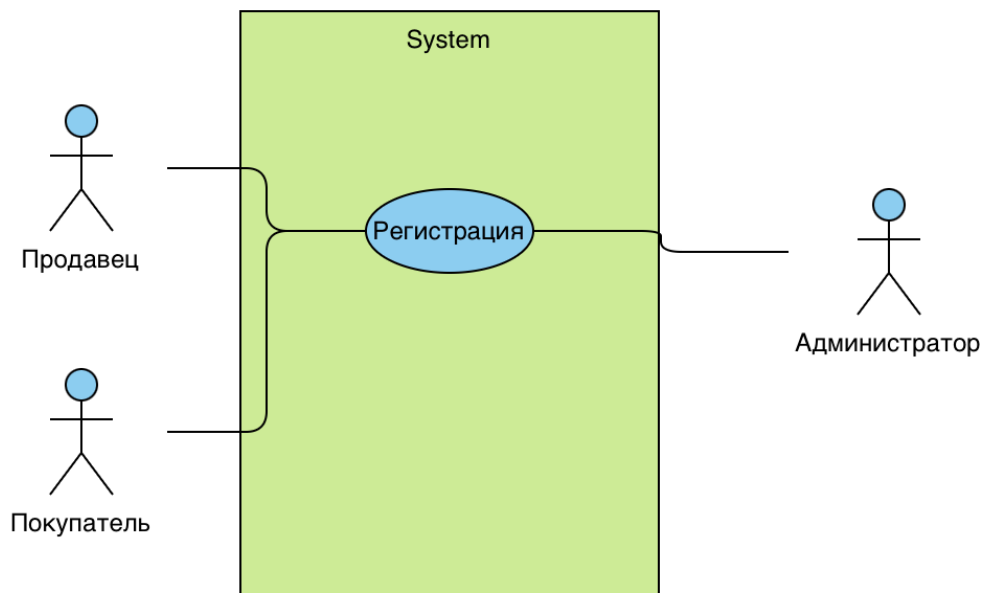
что требование реализовано верно и поведение приложения в точности соответствует требованию.

Документирование требований

1. **Варианты использования (use cases)** представляют собой сценарии взаимодействия пользователя (или пользователей) с программным продуктом для достижения конкретной цели.

Человечками на схеме обозначаются акторы (не опечатка), которые используют систему. Прямоугольником обозначается система, а сам вариант использования (функция) овалом. Это так называемая use case диаграмма.

Подробнее можно прочитать <https://testengineer.ru/что-такое-use-case/>



2. **Пользовательская история (user story)** - общее описание функций программы, написанное как бы от имени пользователя. На современных проектах чаще используют именно ее для документирования требований.

User story создается по шаблону:

"As a [persona], I [want to], [so that]."

«Как [тип клиента], [хочу то-то], [чтобы делать что-то]».

Упрощенные примеры:

- Как Макс, я хочу пригласить друзей, чтобы мы вместе могли пользоваться этим замечательным сервисом.
- Как Саша, я хочу организовать свою работу, чтобы лучше контролировать ситуацию.
- Как менеджер, я хочу видеть, как продвигается работа у моих коллег, чтобы можно было составлять более точные отчеты о наших успехах и неудачах.

Помимо описания самой user story в ней обязательно **содержатся критерии готовности (acceptance criteria)**, в которых и будут отражены конкретные требования к разрабатываемой функции.

Подробнее можно прочитать:

1. <https://www.atlassian.com/ru/agile/project-management/user-stories>
2. <https://testengineer.ru/chto-takoe-user-story-i-kak-ee-pisat>
3. <https://partnerkin.com/blog/articles/chto-takoe-user-story>

Пример :

Тема(EPIC): как быстро авторизоваться в кабинете

История 1: как покупатель, я хочу быстро зарегистрироваться через Google, чтобы быстрее перейти к покупкам.

История 2: как клиент магазина для взрослых, я хочу не указывать личные данные при регистрации, чтобы сохранить свою конфиденциальность.

Примечания: данные с сайта показывают, что большинство пользователей покидают его после открытия формы регистрации; опросы показали, что многие клиенты хотят сохранить свою конфиденциальность даже от владельцев магазина; судя по трафику конкурентов, у которых упрощенная форма регистрации без указания личных данных, их конверсия гораздо выше нашей.

Задачи, за которые выходить не нужно (**acceptance criteria**):

- убрать из формы регистрации все поля, где требуется указывать личные данные — за исключением раздела с вариантами оплаты товаров, а также номера телефона и почты пользователя;
- добавить возможность авторизации через Google или Facebook*;

- добавить к кнопке регистрации ссылку с текстом на политику конфиденциальности.

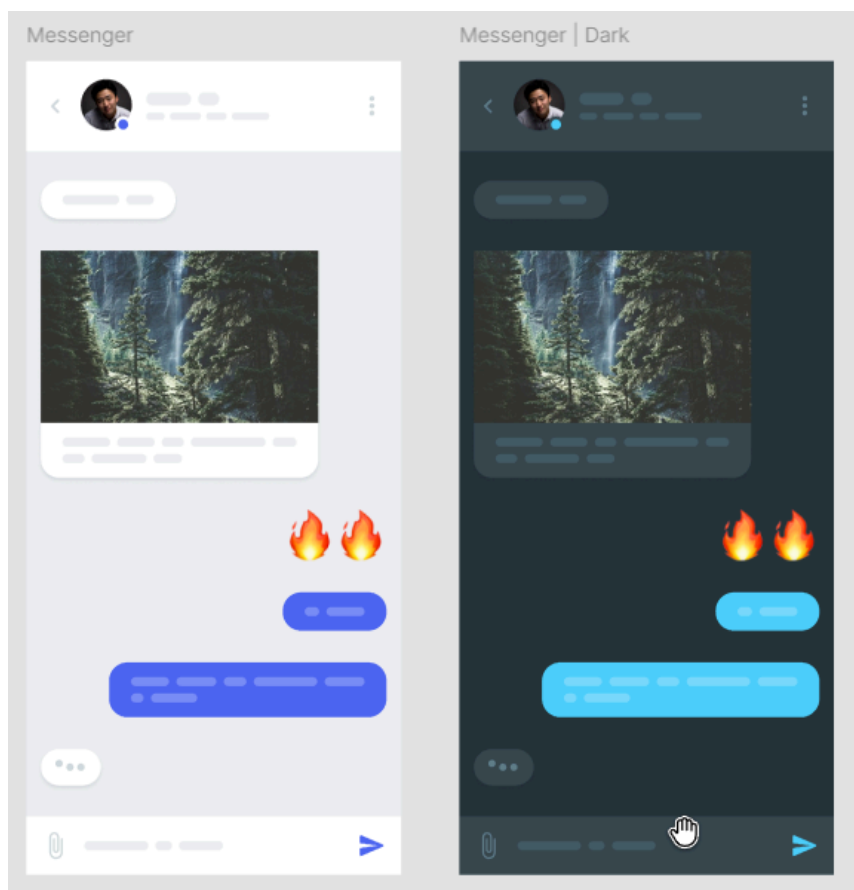
Источник: <https://partnerkin.com/blog/articles/chto-takoe-user-story>

3. **Mockup (макет)** - реалистичная модель нашего приложения, которая помогает показать, как дизайн будет выглядеть на frontend.

В тестировании макеты используются для проверки frontend и именно согласно им проверяем наличие определенных элементов, их размер, стиль, поведение.

пример Mockup

[https://www.figma.com/design/oEF8YOLK20cYwkclngDSg/Mockup-UI-Templates-\(Community\)?node-id=0-1](https://www.figma.com/design/oEF8YOLK20cYwkclngDSg/Mockup-UI-Templates-(Community)?node-id=0-1)



Алгоритм работы с требованиями со стороны тестировщика

Идеальный процесс

1. Требования создают бизнес-аналитики или владельцы продукта.
Обычно они содержат в себе дизайнерские макеты.
2. Требования попадают к тестировщику, который проверяет их на соответствие свойствам качественных требований. Дополнительно в Scrum есть Backlog Refinement, где требования обсуждаются и уточняются командой на отдельном собрании.
3. В случае обнаружения неточностей, требования отправляются бизнес-аналитику на доработку, например, оставляется комментарий в системе.
4. Пункты 2-3 повторяются до тех пор, пока не будут устранены несоответствия. Параллельно требования могут изучать другие участники команды: разработчики, дизайнеры
5. После того как требования согласованы, они берутся в разработку, а тестировщики создают тестовую документацию для дальнейших проверок. Важно: тест-кейсы и чек-листы создаются до того, как функциональность из требования будет разработана. Вспоминаем Shift Left Testing и раннее тестирование.
6. Вносить изменения в требования после начала спринта не рекомендуется и по-хорошему должно быть запрещено.

Неидеальный процесс

1. Анализ требований не всегда осуществляется до этапа разработки, и уточнения могут вноситься уже во время нее
2. Тестовая документация может создаваться уже после разработки функциональности

3. Требования могут вообще не документироваться и быть на проекте в неявном виде

Источники информации:

1. Тестирование программного обеспечения. Базовый курс, стр.18
[-https://svyatoslav.biz/software_testing_book/](https://svyatoslav.biz/software_testing_book/)
2. Библия QA. Модели разработки ПО
https://vladislaveremeev.gitbook.io/qa_bible/sdlc-i-stlc/modeli-razrabotki-po
3. Модели разработки ПО
<https://baikov.dev/software-development-methodologies/#%D0%B6%D0%B8%D0%B7%D0%BD%D0%B5%D0%BD%D0%BD%D1%8B%D0%B9-%D1%86%D0%B8%D0%BA%D0%BB-%D0%BF%D0%BE>